Colecțiile din JDK — Exerciții

Adrian · pornești mereu de la interfață: `List<String> nume = new ArrayList<>();`

Unde scrii: `src/exercitii/Sectiune1Collection.java` … `Sectiune5QueueDeque.java`, `Sectiune6Comparare.java`, `Sectiune7Alegere.java` : metoda cu numele din tabel (datele de pornire sunt deja declarate); `Main` le cheamă pe toate. Cărțile metodelor: **COLECTII.pdf**. Exemple de rulat și citit: `src/exemple/` (`java -cp out exemple.Main`, meniu 1–5).
Ieșirea: elementele unei colecții le afișezi cu `String.join(", ", c)` (la numere, lipești tu elementele cu „, ”); un `boolean` sau un număr cum sunt; excepția o prinzi cu `try/catch (Exception e)` și afișezi `e.getClass().getSimpleName()`. Celula goală = o notezi tu după ce rulezi.

1. Collection<E> → ArrayList, HashSet, LinkedList

```
Collection<String> c = new ArrayList<>();
```

#	Cerința	Ieșire
1.1	**`adaugaCuDuplicate`** `c` e goală. `add` pe rând Ana, Ion, Maria, apoi încă o dată Ana: afișează ce întoarce ultimul `add`, apoi `size()`.	`true` `4`
1.2	**`verificaContine`** `c` = [Ana, Ion, Maria]. Afișează `contains("Ion")`, apoi `contains("Dan")`.	`true` `false`
1.3	**`scoateIonSiZoe`** `c` = [Ana, Ion, Maria]. `remove("Ion")`, apoi `remove("Zoe")` : afișează ce întoarce fiecare, apoi colecția.	
1.4	**`golesteColectia`** `c` = [Ana, Ion, Maria]. `clear()`, apoi afișează `size()`.	`0`
1.5	**`deCeInterfataCollection`** Scrie `afiseaza(Collection<Integer> c)` : afișează `size()`, „: ” și elementele. Cheamă-o cu un `ArrayList`, un `HashSet` și un `LinkedList`, toate umplute din `v`. Ce ordine are fiecare ieșire?	

2. List<E> → ArrayList, Arrays.asList

```
List<String> l = new ArrayList<>();
```

#	Cerința	Ieșire
2.1	**`puneDanPePrimaPozitie`** `l` = [Ana, Ion, Maria]. Pune „Dan” pe prima poziție (`add` cu poziție); afișează lista.	`Dan, Ana, Ion, Maria`
2.2	**`pozitiaLuiMariaSiZoe`** `l` = [Ana, Ion, Maria]. Afișează `indexOf("Maria")`, apoi `indexOf("Zoe")`.	`2` `-1`
2.3	**`inlocuiesteIonCuEvaSiScoatePrimul`** `l` = [Ana, Ion, Maria]. Înlocuiește „Ion” cu „Eva” (`set`), apoi scoate elementul de pe poziția 0 (`remove` cu poziție); afișează lista.	`Eva, Maria`
2.4	**`citesteDupaUltimaPozitie`** `l` = [Ana, Ion, Maria]. Citește `get(3)` într-un `try/catch`; afișează numele excepției.	
2.5	**`deCeInterfataList`** Scrie `mareste(List<Integer> l)` (`add(4)`, apoi afișează lista) și `schimba(List<Integer> l)` (`set(0, 9)`, apoi afișează lista). Cheamă-o pe fiecare cu un `ArrayList` și cu `Arrays.asList(1, 2, 3)`, ambele {1, 2, 3}; `mareste` pe `Arrays.asList`, în `try/catch`.	

3. Set<E> → HashSet, LinkedHashSet, TreeSet

```
Set<String> s = new HashSet<>();
```

Datele de pornire sunt deja declarate în starter; 3.2 și 3.3 pornesc de la un `LinkedHashSet`, ca ordinea afișată să fie fixă.

#	Cerința	Ieșire
3.1	**`adaugaAnaIonAnaMaria`** `s` e goală. `add` pe rând Ana, Ion, Ana, Maria: afișează ce întoarce fiecare `add`.	
3.2	**`reuniuneCuIonSiEva`** `s` = {Ana, Ion, Maria}. `addAll` cu [Ion, Eva]; afișează mulțimea.	`Ana, Ion, Maria, Eva`
3.3	**`intersectieCuIonMariaZoe`** `s` = {Ana, Ion, Maria}. `retainAll` cu [Ion, Maria, Zoe]; afișează mulțimea.	`Ion, Maria`
3.4	**`scoateIonSiNumara`** `s` = {Ana, Ion, Maria}. `remove("Ion")`; afișează `contains("Ion")`, apoi `size()`.	`false` `2`
3.5	**`deCeInterfataSet`** Scrie `afiseazaMultime(Set<String> s)`: afișează `size()`, „: ” și elementele. Cheamă-o cu un `HashSet`, un `LinkedHashSet` și un `TreeSet`, toate umplute din `v`. Ce ordine are fiecare ieșire?	

4. Map<K,V> → HashMap, LinkedHashMap, TreeMap

```
Map<String, Integer> d = new HashMap<>();
```

#	Cerința	Ieșire

4.1	**`adaugaSiSuprascrie`** `d` e goală. `put("Ana", 9)`, `put("Ion", 7)`, apoi `put("Ion", 10)`: afișează ce întoarce ultimul `put`, apoi `size()` și `get("Ion")`.	`7`
		`2`
		`10`

4.2	**`cautaCuGetSiGetOrDefault`** `d` = {Ana: 9, Ion: 7}. Afișează `get("Dan")`, apoi `getOrDefault("Dan", 0)`, apoi `get("Ana")`.	`null`
		`0`
		`9`

4.3	**`putIfAbsentSiNullPeDan`** `d` = {Ana: 9, Ion: 7}. Afișează ce întoarce `putIfAbsent("Ana", 5)`, apoi `get("Dan")`. Apoi `int x = d.get("Dan")` într-un `try/catch`: afișează `x` sau numele excepției.	

4.4	**`scoateAnaDeDouaOri`** `d` = {Ana: 9, Ion: 7}. `remove("Ana")` de două ori, apoi `containsKey("Ion")`; afișează cele 3 rezultate.	`9`
		`null`
		`true`

4.5	**`deCeInterfataMap`** Scrie `afiseazaDictionar(Map<String, Integer> d)`: pentru fiecare pereche afișează `[cheie, valoare]`, perechile separate prin „, ”, pe un rând. Cheamă-o cu un `HashMap`, un `LinkedHashMap` și un `TreeMap`, toate umplute din `date`. Ce ordine are fiecare ieșire?	

5. Queue<E> și Deque<E> → ArrayDeque

O singură clasă, `ArrayDeque`, acoperă și coada, și stiva; alegi interfața după ce vrei să faci.

```
Queue<String> q = new ArrayDeque<>();
Deque<String> s = new ArrayDeque<>();
```

#	Cerința	Ieșire

5.1	**`servesteUnClient`** `q` = [Ana, Ion, Maria] (primul = Ana, puse cu `offer`). `poll()`, apoi `peek()`, apoi `size()`; afișează cele 3 rezultate.	`Ana`
		`Ion`
		`2`

5.2	**`stivaPopPeekSiParcurgere`** `s`: `push` pe rând Ana, Ion, Maria. `pop()`, apoi `peek()`, apoi toată stiva cu `String.join`; afișează cele 3.	`Maria`
		`Ion`
		`Ion, Ana`

5.3	**`scoateToataCoada`** `q` = [Ana, Ion, Maria]. Cu `while (!q.isEmpty())` scoate și afișează fiecare client.	`Ana`
		`Ion`
		`Maria`

5.4	**`scoateDinGol`** O coadă goală: afișează `poll()`, apoi `peek()`, apoi (în `try/catch`) `remove()`. O stivă goală: `pop()`, în `try/catch`. Pentru excepții afișează numele ei.	

6. Comparable & Comparator → ordinea naturală și ordinea din afară

```
class Elev implements Comparable<Elev> { public int compareTo(Elev altul) { ... } }
class ComparatorDupaNume implements Comparator<Elev> { public int compare(Elev a, Elev b) { ... } }
```

Datele de pornire sunt declarate în starter. Fiecare comparator îl scrii într-un fișier nou în `src/exercitii/`, cu numele din tabel; la 6.1 modifici chiar clasa `Elev`. Elevii sunt afișați ca `nume(nota)`.

#	Cerința	Ieșire
6.1	**`faElevulComparabil`** Fă `Elev` `Comparable<Elev>`: `compareTo` compară după notă, crescător (folosește `Integer.compare`). Sortează `lista` cu `Collections.sort(lista)` și afișează-o.	`[Ion(6), Dan(7), Ana(9), Maria(10)]`
6.2	**`treeSetAreNevoieDeComparabil`** După ce ai făcut 6.1 (`Elev` comparabil): pune elevii din `elevi` într-un `TreeSet` și afișează-l. Apoi pune `masina` (clasa `Masina` nu e comparabilă) într-un `TreeSet<Masina>`, în `try/catch`: afișează numele excepției.	
6.3	**`comparatorDupaNume`** Scrie clasa `ComparatorDupaNume` (`Comparator<Elev>`, metoda `compare`): ordinea alfabetică a numelor. Sortează `lista` cu `Collections.sort(lista, comparator)` și afișează-o.	`[Ana(9), Dan(7), Ion(6), Maria(10)]`
6.4	**`comparatorDupaNotaDescrescator`** Scrie clasa `ComparatorDupaNotaDescrescator`: nota cea mai mare prima. Sortează `lista` cu `lista.sort(comparator)` și afișează-o.	`[Maria(10), Ana(9), Dan(7), Ion(6)]`
6.5	**`treeMapCuComparator`** Scrie clasa `ComparatorDupaLungime` (`Comparator<String>`): după lungime, iar la lungimi egale alfabetic. Pune fiecare cheie din `chei` cu lungimea ei ca valoare în două `TreeMap`: unul `new TreeMap<>(comparator)`, unul fără comparator; afișează ambele hărți.	`{ou=2, unt=3, lapte=5, paine=5}` `{lapte=5, ou=2, paine=5, unt=3}`
6.6	**`compareToEgalInseamnaDuplicat`** Cu `compareTo` de la 6.1, pune `ana`, apoi `vlad` într-un `TreeSet`: afișează ce întoarce fiecare `add`, apoi mulțimea. Pune-i apoi într-un `HashSet` și afișează `size()`. De ce?	

7. Ce colecție alegi — aplici

Pentru fiecare situație scrii colecția pe care o alegi (coloana „Colecția aleasă”), codul în metoda ei, apoi notezi ieșirea (un rând, valorile separate prin „ ”).

#	Situația	Colecția aleasă	Ieșire
7.1	`situatiaVizitatori` La intrare trec, pe rând: Ana, Ion, Ana, Maria, Ion. Afișează câți vizitatori **diferiți** au fost.		
7.2	`situatiaIstoric` Butonul „Înapoi”: ai deschis paginile A, B, C (în ordinea asta). Apeși „Înapoi” de două ori: afișează paginile în ordinea întoarcerii, pe un rând.		
7.3	`situatiaTelefoane` Agenda: Ana → 0711, Ion → 0722. Caută „Ion”, apoi „Dan”; afișează numerele pe un rând (cuvântul `lipsa` dacă nu există).		
7.4	`situatiaTemperaturi` Exact 5 zile, temperaturile 12, 8, 15, 9, 11. Sortează-le; afișează lista sortată și, după ea, poziția lui 15 **în lista sortată**.		
7.5	`situatiaGhiseu` Clienții la ghișeu, serviți în ordinea sosirii: Ana, Ion, Maria. Servește 2; afișează-i în ordinea servirii, pe un rând.		
7.6	`situatiaNumere` Numerele 5, 3, 8, 3, 1. Afișează cel mai mic, cel mai mare, apoi toate numerele **fără dubluri**, crescător; totul pe un rând.		
7.7	`situatiaNote` Notele 9, 6, 10, 7, în ordinea asta, pe poziții. Pune 8 pe poziția 1, scoate elementul de pe poziția 3, sortează; afișează lista pe un rând.		
7.8	`situatiaScoruri` Scorurile Zoe = 5, Ana = 9, Ion = 7, cu numele mereu în ordine alfabetică. Afișează **a doua** cheie în ordine.		
7.9	`situatiaMetrou` Pasagerii la coadă: Ion, Maria. Pune-o pe Ana la început și pe Dan la sfârșit, apoi scoate ultimul; afișează rândul pe un rând.		
7.10	`situatiaClasament` Scorurile Zoe = 5, Ana = 9, Ion = 7, căutate după nume. Afișează-le în ordinea alfabetică a numelor, ca `nume=scor`, pe un rând.		